

Module 4 — Integration: KPI Dashboard

Build a descriptive analytics report with a KPI framework for the Amman Digital Market — the same business domain from Module 3's ETL pipeline. Extract data from PostgreSQL, define KPIs, validate them with statistical tests, create publication-quality visualizations, and write an executive summary.

Due: Day 5 (Thursday) end of day. Resubmissions are accepted through Saturday of the assignment week.

Setup

1. **Accept the assignment:** [Accept the M4 Integration assignment](#)
2. Clone your repo and open it in VS Code.
3. Start your Postgres container (reuse from M3, or create a new one):

```
docker run -d --name postgres-m4-int \
-e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres \
-e POSTGRES_DB=amman_market \
-p 5432:5432 -v pgdata_m4_int:/var/lib/postgresql/data \
postgres:15-alpine
```

4. Create the database and load the schema + seed data:

```
psql -h localhost -U postgres -d amman_market -f schema.sql
psql -h localhost -U postgres -d amman_market -f seed_data.sql
```

5. Verify: `psql -h localhost -U postgres -d amman_market -c "\dt"` should show 4 tables.
6. Install dependencies: `pip install -r requirements.txt`

If you still have the M3 integration database running, you can reuse it — the schema and seed data are the same.

The Database

Amman Digital Market is a fictional online marketplace with four tables:

Table	Description	Rows	Key columns
customers	Registered users	~100	customer_id (PK), customer_name, email, city, registration_date
products	Product catalog	~30	product_id (PK), product_name, category, unit_price
orders	Customer orders	~500	order_id (PK), customer_id (FK), order_date, status
order_items	Line items per order	~1200	item_id (PK), order_id (FK), product_id (FK), quantity

Data quality reminders (from M3):

- Some orders have `status = 'cancelled'` — exclude from analytics
- Some customers have `NULL` city values
- Some `order_items` have `quantity > 100` — likely data entry errors; exclude

Tasks

Complete the implementation of `analysis.py` — the starter file in your repo. The file contains function signatures with docstrings — you implement the logic.

Task 1 — Data Extraction

Connect to the Amman Digital Market Postgres database using SQLAlchemy. Extract all four tables into DataFrames.

```
from sqlalchemy import create_engine

engine = create_engine("postgresql+psycopg://postgres:postgres@localhost:5432/amman_market")
```

This reuses your M3 ETL skills — extraction should take minutes, not hours. Filter out cancelled orders and suspicious quantities (quantity > 100) as part of your data preparation.

Task 2 — KPI Framework Design

Define **5 KPIs** for the Amman Digital Market. Document them in `kpi_framework.md` using this structure for each KPI:

Element	Your Content
Name	Clear, specific (e.g., "Monthly Revenue Growth Rate")
Definition	Precisely what is being measured
Formula	The computation (e.g., "(current_month - prev_month) / prev_month")
Data source	Which tables/columns are used
Baseline value	Current value computed from the data
Interpretation	What this number means for the business

Requirements:

- At least **2 KPIs** must involve time-based grouping (e.g., monthly revenue trend, weekly order volume, month-over-month growth)
- At least **1 KPI** must involve cohort segmentation (e.g., revenue by city, average order value by product category, customer lifetime value by registration cohort)

Think about what a market manager would need to track. Good KPIs are actionable — a change should trigger a business response.

Task 3 — Statistical Validation

For at least **2 of your KPIs**, run a statistical test to validate the observed pattern:

- Example: "Is the revenue difference between Amman and Irbid statistically significant?" — independent samples t-test
- Example: "Is there an association between product category and customer city?" — chi-square test
- Example: "Does average order value differ across product categories?" — ANOVA

For each test, report:

- The hypothesis (H_0 and H_1)
- The test used and why it is appropriate
- The test statistic and p-value
- A plain-language interpretation
- Effect size (Cohen's d or Cramer's V, as appropriate)

Document the results in `kpi_framework.md` alongside the relevant KPI.

Task 4 — KPI Visualizations

Add visualization code to `analysis.py` to generate a multi-chart report visualizing all 5 KPIs. Save all charts to `output/` as PNG files.

Requirements:

- At least **one multi-panel figure** using `fig, axes = plt.subplots()`
- At least **one Seaborn statistical plot** (e.g., `boxplot`, `heatmap`, `violinplot`)
- Every chart must have: a descriptive title (state the finding), axis labels, and a legend where appropriate
- Use colorblind-safe palettes (`sns.set_palette('colorblind')` or Matplotlib's `viridis`)

Suggested chart types (choose based on your KPIs):

- Line chart for time-based KPIs (monthly trends)
- Bar chart for comparisons (revenue by city, category)
- Box plot for distribution comparisons (order value by category)
- Heatmap for cross-tabulations (category × city revenue)

Task 5 — Executive Summary

Write `EXECUTIVE_SUMMARY.md` — a one-page summary presenting your top 3 findings. This is written for a **non-technical business audience** (a market manager, not a data scientist).

Structure:

1. **Overview:** One paragraph describing the analysis scope and data
2. **Finding 1:** The insight, supported by a KPI value and statistical test result. Reference the chart file.
3. **Finding 2:** Same structure.
4. **Finding 3:** Same structure.
5. **Recommendations:** 2–3 actionable recommendations based on your findings. Each recommendation should connect directly to a KPI.

Writing guidance:

- Lead with the finding, not the method. "Amman generates 2.5x more revenue than Irbid" — not "A t-test was conducted on revenue by city."
- Include numbers but keep them simple. "Revenue grew 15% month-over-month in Q4" — not "The mean monthly revenue increased from 4,532.17 JOD to 5,212.00 JOD."
- Reference chart files by name so the reader can look them up.

How This Assignment Is Graded

Your TA grades this assignment on 5 dimensions: KPI Framework, Statistical Validation, Visualizations, Executive Summary, and Code Quality. Your code is the baseline. What matters more on this assignment is your **analytical judgment**, not your code.

AI screening: Before scoring, your TA runs every submission through an AI screening tool that checks for generic, AI-generated content. Submissions flagged as AI-generated receive a **0 with resubmission required**.

This is not about catching you — it is about the learning objective. The point of this assignment is for *you* to decide which KPIs matter, interpret what the statistics mean for *this* business, and write recommendations that a market manager could act on. An AI can produce a technically correct submission in minutes, but it cannot teach you to think analytically. If your KPIs are things any AI would suggest for any e-commerce dataset, your executive summary reads like a consulting template, or your statistical interpretations are textbook copy-paste — the screen will flag it, and you will need to redo the work.

Use your brain. Look at the actual data. Let the numbers surprise you. The best submissions come from learners who ran a query, noticed something unexpected, and followed the thread.

Tests

Write your tests in `tests/test_analysis.py` — the starter file has 3 test functions with docstrings describing what each should verify. Run with `pytest tests/ -v`.

Submit

1. Create branch `integration-4-kpi-dashboard`
2. Commit: `analysis.py`, `kpi_framework.md`, `EXECUTIVE_SUMMARY.md`, `output/` directory (all chart PNGs), `tests/test_analysis.py`, and any supporting code
3. Open a PR to `main`
4. Your PR description must include:
 - The names of your 5 KPIs
 - One statistical test result (test used, statistic, p-value, interpretation)
 - A note about your database connection (confirm Postgres was used)
5. Paste your PR URL into TalentLMS → Module 4 → Integration 4 to submit this assignment.

Challenge Extensions

These are optional extensions for learners who complete the base requirements.

Tier 1 — Interactive Dashboard with Plotly

Convert your static KPI charts to **interactive Plotly visualizations** using `plotly.express`. Save as `output/dashboard.html` — a standalone HTML file with hover tooltips, zoom, and filter.

Plotly is covered conceptually in the reading. This tier requires applying it:

- `px.bar()`, `px.line()`, `px.scatter()` for your KPI charts
- `fig.update_layout()` for titles and labels
- `fig.write_html()` for the standalone output

Tier 2 — Automated KPI Monitoring Script

Build `kpi_monitor.py` that:

- Connects to the database and computes all 5 KPIs
- Reads baseline thresholds from a `config.json` file
- Outputs **green/yellow/red status** per KPI based on threshold comparison
- Uses Plotly `Indicator` traces to create gauge visualizations (actual vs. target)
- Includes dropdown filters (city, category, date range) using `plotly.graph_objects` with `updatemenus`
- Write **pytest tests**: mock the database connection, verify threshold logic, verify output format

Tier 3 — Multi-Page Analytical Report with Plotly Dash

Build a **Plotly Dash application** with multiple pages:

- **Page 1:** KPI overview with gauge indicators
- **Page 2:** Time-series deep dive with date range selector
- **Page 3:** Cohort comparison with drill-down capability

Implement **callback functions** for cross-filtering (selecting a city on Page 1 filters data on Pages 2 and 3). Deploy locally with `python app.py` and document the architecture.

Requires: `pip install dash` (self-managed for this challenge tier only — not part of the course standard environment).